Facebook (Meta) Gizmo - From Cardboard Prototypes to Secure Corporate Devices

Davide Guerri
Security Engineering Leader @ Google | Vulnerability Management + Adjunct Professor @ Sapienza Università di Roma | Cybersecurity, Ethical Hacking + Sommelier @ AIS (Associazione…

August 14, 2025

Back in my early years at Facebook (now Meta), I worked on something that was both challenging and surprisingly fun.

The most unexpected part? I would never have guessed I'd end up working on it at all.

I was part of a team called Collaboration Engineering. Our focus was building collaboration tools for internal use, starting with videoconferencing systems. I was one of only two production engineers embedded in a software engineer–heavy team. The big goal was to build the next generation of conferencing software that could scale to hundreds of thousands of employees around the world.

In my early days on the team, I was given a potential project related to provisioning, via Chef, on an embedded device. This device was meant to replace a growing number of Android tablets running custom kiosk software. Those tablets were used to show the meeting room booking calendar and let people book a room right before walking in.

The challenges with the original approach were clear. We had to ensure continuous delivery of the core application, manage a mix of user-space and kernel-space custom software, and handle operating system upgrades. Doing that with Chef was not going to be pleasant.

Fortunately, my manager at the time was open-minded enough to consider an alternative design I proposed.

Chef works well in a tightly controlled, standardized environment like a datacenter (though I could tell you some war stories about thousands of thousands of "ghost" servers forgotten in Chef's kitchen). But a corporate environment, where devices are physically accessible to employees, was another matter entirely. Think about the kinds of secrets these devices might store: passwords, keys, tokens. The previous Android-based implementation had actually been used by the Facebook Red Team to reach production systems. Even worse, some of these devices had been found on eBay for a few dollars.

The idea was simple. Instead of provisioning devices with a base OS and updating them periodically through Chef, we would provision the entire disk image as a golden image on boot. Everything would be included: the core app, a Chromium browser to run it, all kernel and user-space drivers configured exactly as in development and staging. The image would be read-only. New versions would be deployed simply by rebooting the device.

Of course, we still had to address some serious security requirements. The threat model for these devices included:

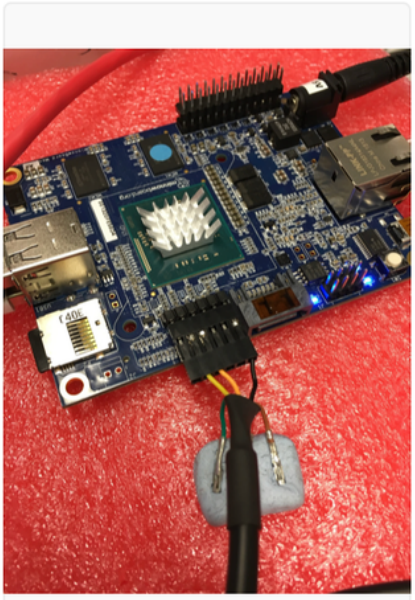
- A stolen device with its contents analyzed (*)
- Interception of device traffic (passive or MITM)
- Device impersonation to download secrets, configuration, or non-public software
- Unauthorized access to a running device
  So, at minimum, our provisioning system had to:

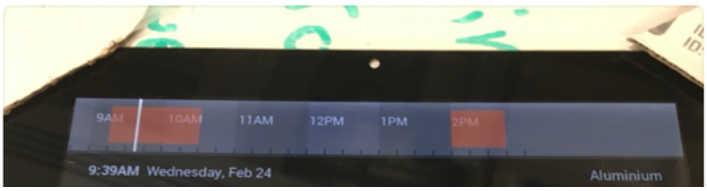
- Identify each device uniquely so the right application and settings could be delivered

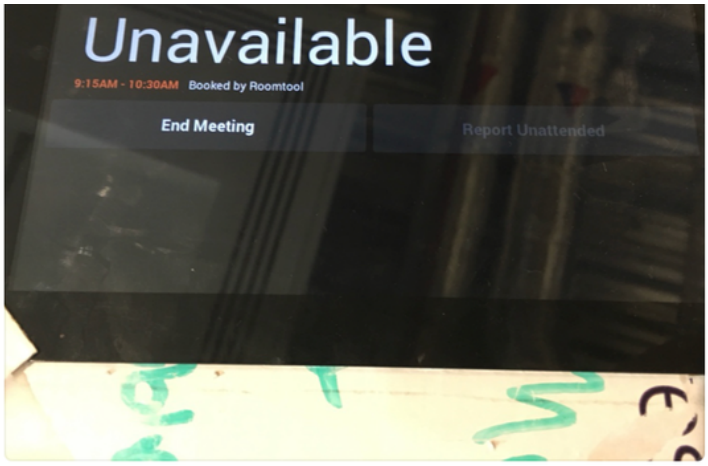
- Deliver secrets securely
- Avoid storing configuration, secrets, or applications locally
- Ensure golden images containing non-public software were only provisioned to trusted devices, even over an untrusted network
  Around that time (this was 2016), I had been reading about Trusted Platform Module (TPM) technology and its role in secure boot and disk encryption. I knew TPMs could do both symmetric and asymmetric encryption, and I wondered if we could use them as the security foundation for our devices. That would have been perfect, but TPMs on embedded hardware were rare back then, and most were still version 1.0, which lacked the cryptographic features we needed. We required 2.0.

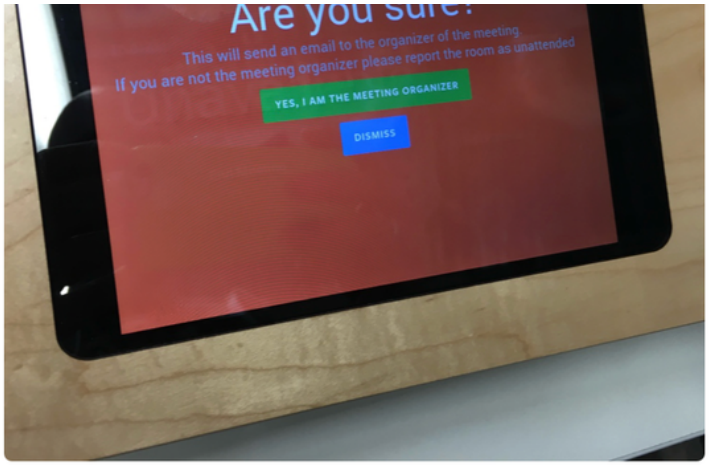
  One of the big mottos at Facebook then was, “What would you do if you were not afraid?” Our provisioning idea was solid, and the team realized it could be applied to far more than just the meeting room tablets. We could use it for Wayfinders (interactive maps in Meta offices), videoconferencing control panels, monitoring dashboards, and more.

  So, we started prototyping using one of the few devices that could be extended with a firmware TPM 2.0: the MinnowBoard Max (http://minnowboard.outof.biz/MinnowBoard_MAX.html). At first, the prototypes were rough, built with cardboard and tape. Later we got nice wooden cases with embedded LEDs!

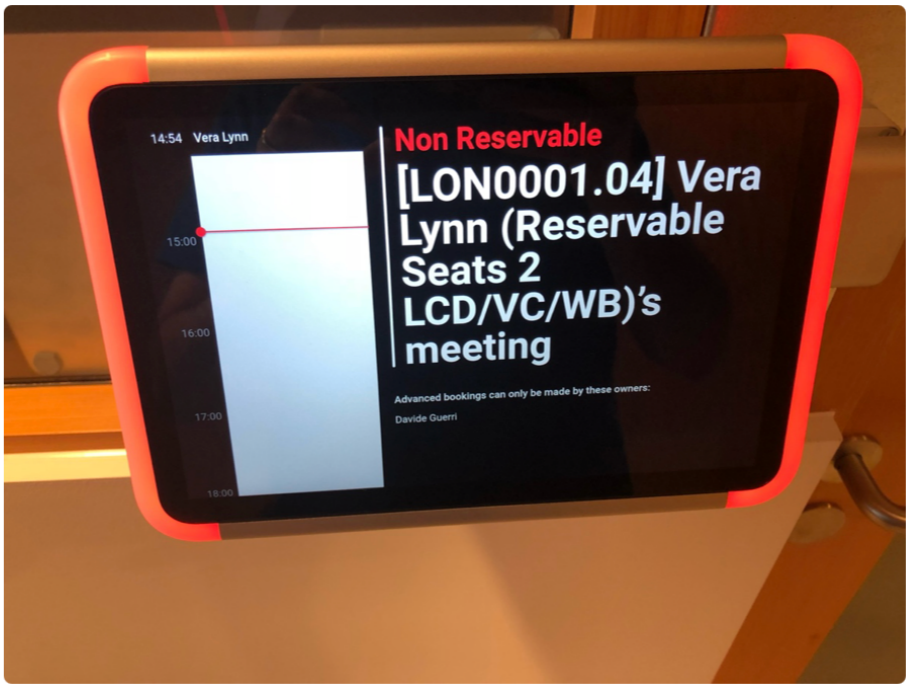








As the prototype matured and proved viable, we secured enough funding to hire a couple of electronics engineers and design fully custom hardware and mechanics from scratch. That is how Gizmo was born: https://andrea-locatelli.com/facebook-roomtool.



It was not perfect from day one. Gizmo went through several iterations and improvements. But the provisioning design was flexible enough to support quick, safe experimentation. The golden image approach made it easy to canary new OS and app versions, and the provisioning system let us roll back instantly if needed. Integration with corporate network automation even allowed us to power-cycle devices automatically when they failed health checks (yes, Gizmo used Power over Ethernet).

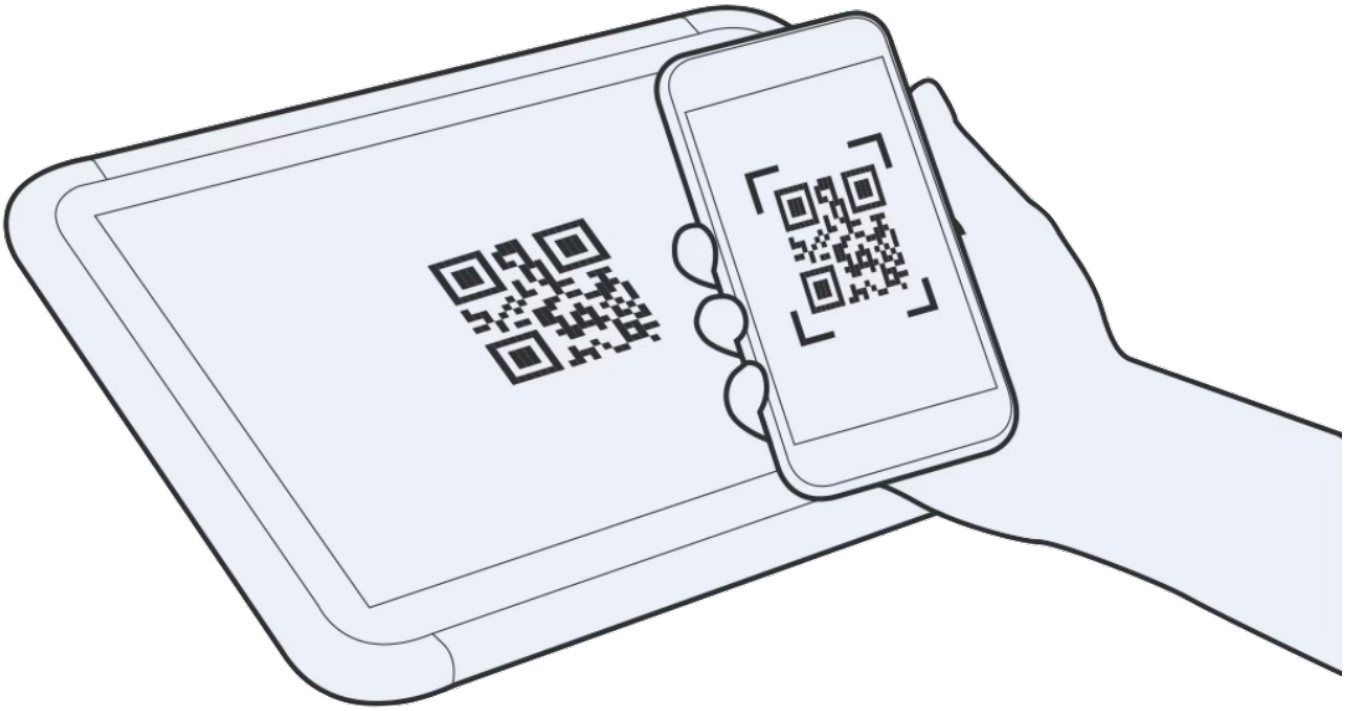
Here is how it worked in practice.

Each Gizmo could be in one of two states: unprovisioned or provisioned.

Devices arrived from the factory (Quanta Computing at the time) with custom firmware we wrote. Initially, this was a UEFI (TianoCore EDK II) customization, stored in the NIC's option ROM and signed to prevent tampering. Later, we switched to a complete Coreboot implementation with the same logic.

The firmware booted from the network using HTTPS, downloading a kernel and initramfs that contained a custom second-stage bootloader. The bootloader identified the device to the provisioning system. If it was unprovisioned, the system sent a rootfs and kernel containing the enrollment image. The device then kexec'ed into that image.

The enrollment image displayed a QR code that refreshed every few minutes. An operator would scan it with a custom application, log in with corporate credentials, and enter the device location, owner, and purpose (VC control panel, Wayfinder, RoomTool, etc.).



During enrollment, the device generated an RSA key pair and sent the public key to Facebook's backends via the operator application.

Once enrolled, the device rebooted. Now, during boot, it could be securely identified, and the backend could deliver the proper OS image, encrypted with a unique symmetric key for that image version. The symmetric key was sent to the device inside a public-key encryption envelope.

After boot, the same encryption mechanisms were used to send device- and application-specific settings and secrets. Among those secrets, we also sent a certificate and a key to authenticate the devices, via 802.11x, onto a network VLAN specifically used for corporate devices.

--

(*) And yes, in case you are wondering, someone actually stole a Gizmo. They still could not extract any protected information. https://web.archive.org/web/20230123024913/https://twitter.com/Foone/status/1310377930661351424 (the original post is now protected).